

API TESTING REPORT

Postman-Based Testing of Users & Tasks REST API

Prepared with Postman

Date: Aug/6/2026

Prepared by Nour Albzoor

1. Introduction

This report documents the functional testing of a REST API that manages two main resources: Users and Tasks. The testing was carried out using Postman, a widely used tool for building, sending, and automating API requests. The goal of this testing effort is to verify that each API endpoint behaves as expected under both valid and invalid conditions, covering all core HTTP methods: GET, POST, PUT, and DELETE.

The API documentation was reviewed using Swagger before testing began, in order to understand the available endpoints, required parameters, and expected response formats. All requests were then organized and executed in a structured Postman Collection, divided into two folders: one for the Users endpoints and one for the Tasks endpoints.

2. Objective

The objective of this testing activity is to:

- Validate that all API endpoints return the correct HTTP status codes for valid requests.
- Validate that the API correctly handles invalid input and non-existent resources (error handling).
- Confirm that CRUD operations (Create, Read, Update, Delete) work correctly for both the Users and Tasks resources.
- Document evidence of each test case using Postman response screenshots.

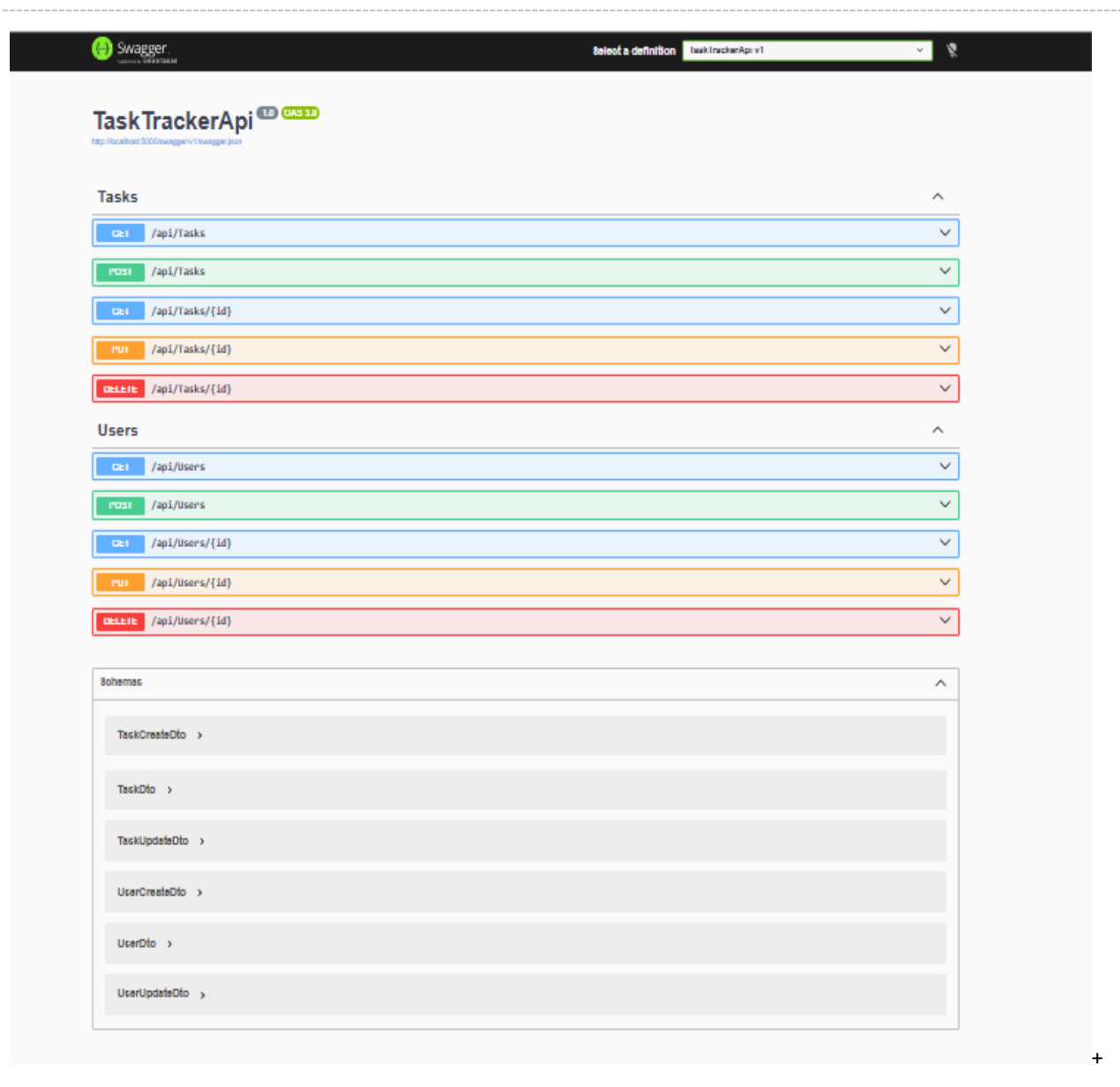
3. Testing Environment

3.1 Tools Used

- Postman — for sending requests and validating responses.
- Swagger — for reviewing the API's documented endpoints before testing.

3.2 API Documentation (Swagger)

Before testing, the Swagger was reviewed to identify all available endpoints, request bodies, and expected responses for both the Users and Tasks resources. A screenshot of the Swagger UI is included below for reference.



3.3 Base URL Configuration

Instead of hardcoding the API's base URL into every request, a Postman environment variable named `{{baseUrl}}` was created and used throughout the collection. This approach makes the collection reusable across different environments (e.g., local, staging, production) by simply updating the variable's value, without needing to edit each request individually.

Example of variable usage in a request URL:

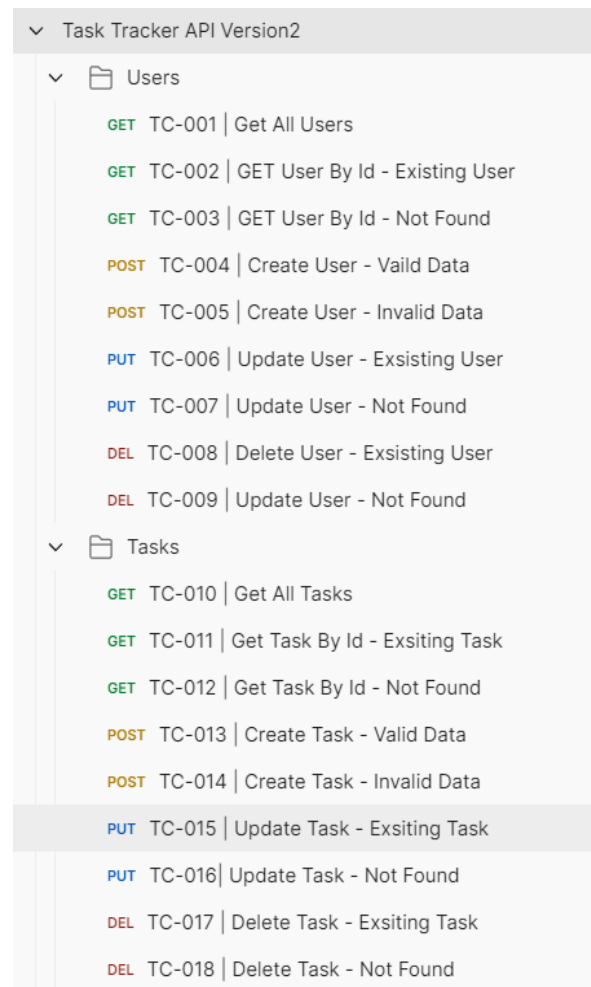
```
{{baseUrl}}/api/users
```

4. Postman Collection Structure

All API requests were organized inside a single Postman Collection, split into two folders for clarity and maintainability:

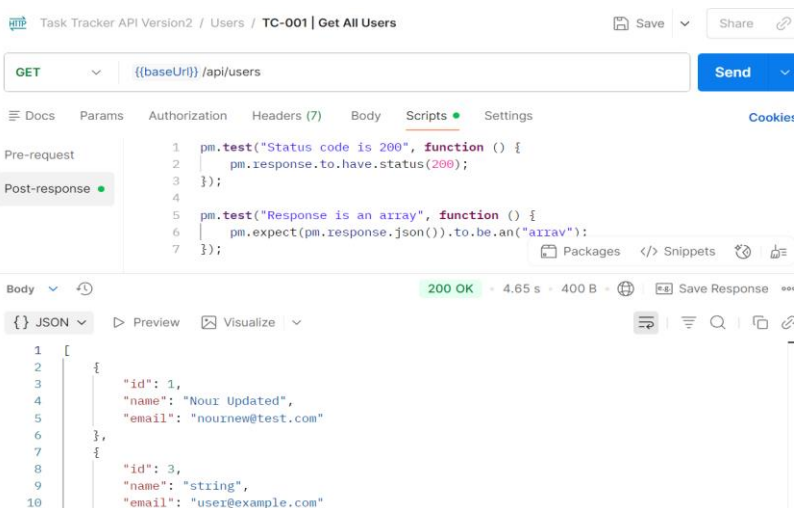
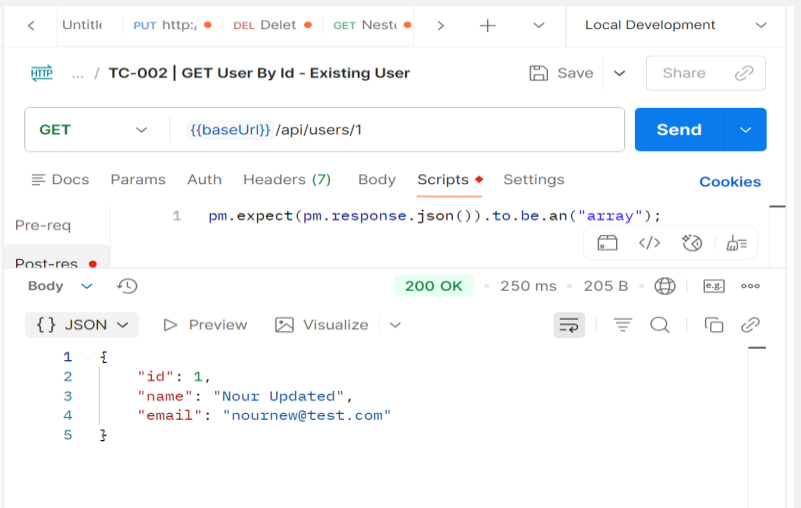
- Users Folder — contains all requests related to the Users resource (GET, POST, PUT, DELETE).
- Tasks Folder — contains all requests related to the Tasks resource (GET, POST, PUT, DELETE).

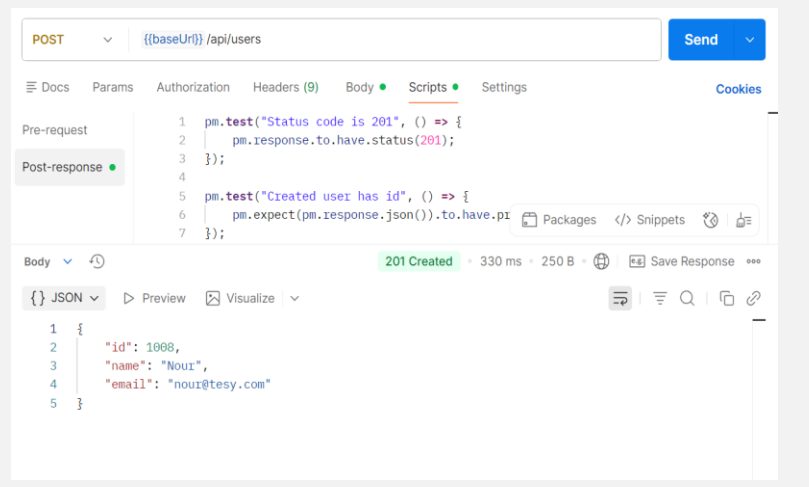
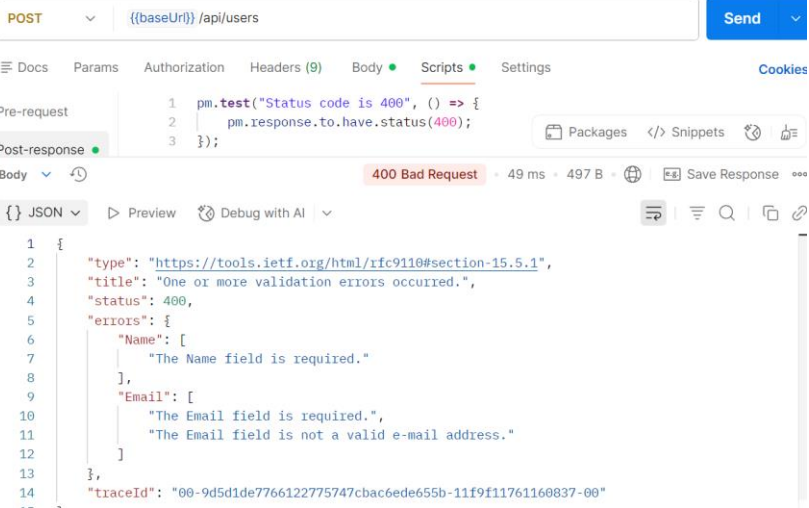
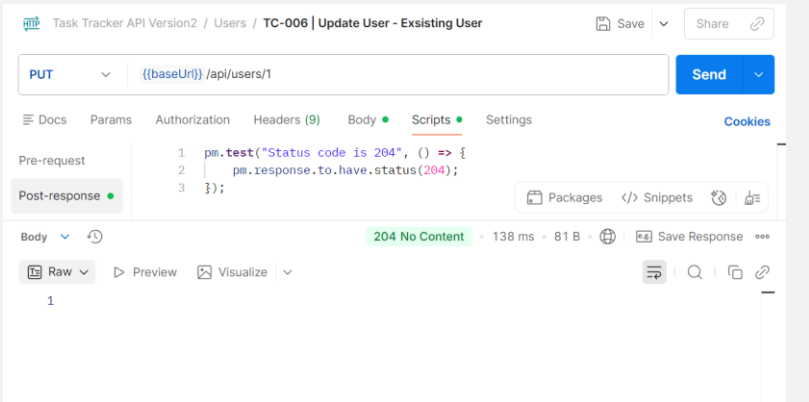
This structure keeps the collection easy to navigate and mirrors the logical separation of resources in the API itself. A screenshot of the full collection tree is included below.



5. Test Cases — Users Endpoint

The table below lists all test cases executed against the `/api/users` endpoint, covering successful requests as well as expected error scenarios (invalid ID, invalid body).

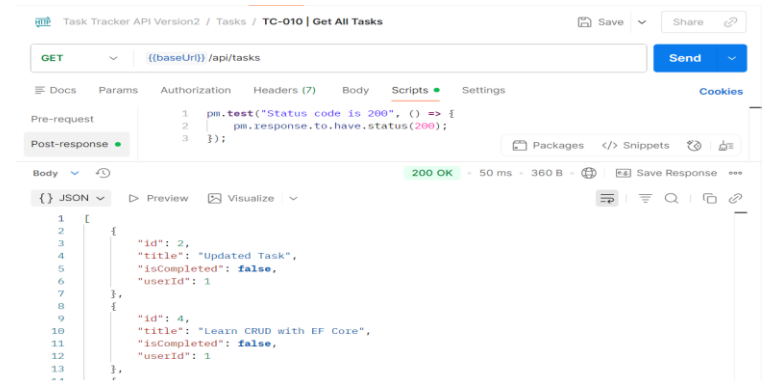
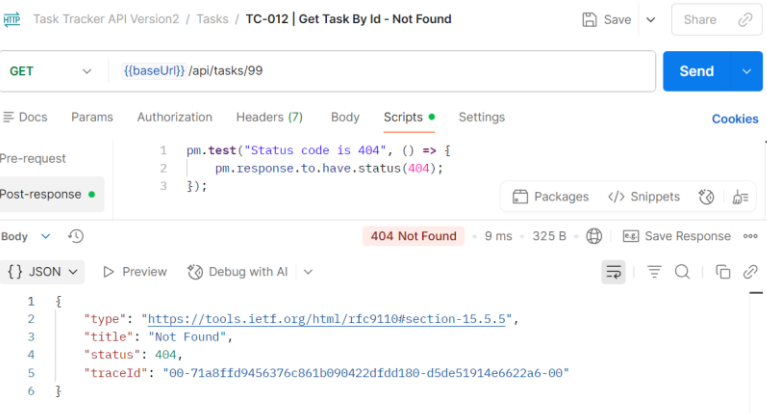
Test ID	Method	Endpoint	Expected Result	Screenshot
TC-001	GET	/api/users	200 OK	 Task Tracker API Version2 / Users / TC-001 Get All Users GET {{baseUrl}} /api/users Pre-request <pre>1 pm.test("Status code is 200", function () { 2 pm.response.to.have.status(200); 3 });</pre> Post-response <pre>5 pm.test("Response is an array", function () { 6 pm.expect(pm.response.json()).to.be.an("array"); 7 });</pre> Body <pre>1 [2 { 3 "id": 1, 4 "name": "Nour Updated", 5 "email": "nournew@test.com" 6 }, 7 { 8 "id": 3, 9 "name": "string", 10 "email": "user@example.com" 11 }</pre>
TC-002	GET	/api/users/1	200 OK	 ... / TC-002 GET User By Id - Existing User GET {{baseUrl}} /api/users/1 Pre-req <pre>1 pm.expect(pm.response.json()).to.be.an("array");</pre> Post-res Body <pre>1 { 2 "id": 1, 3 "name": "Nour Updated", 4 "email": "nournew@test.com" 5 }</pre>
TC-003	GET	/api/users/99	404 Not Found	 Task Tracker API Version2 / Users / TC-003 GET User By Id - Not Found GET {{baseUrl}} /api/users/99 Pre-request <pre>1 pm.test("Status code is 404", () => { 2 pm.response.to.have.status(404); 3 });</pre> Post-response Body <pre>1 { 2 "type": "https://tools.ietf.org/html/rfc9110#section-15.5.5", 3 "title": "Not Found", 4 "status": 404, 5 "traceId": "00-b02c92d2eb4e84475c00363594e625e6-48ead3a33b8ffad9-00" 6 }</pre>

Test ID	Method	Endpoint	Expected Result	Screenshot
TC-004	POST	/api/users	201 Created	
TC-005	POST	/api/users (Invalid Data)	400 Bad Request	
TC-006	PUT	/api/users/1	204 No Content	

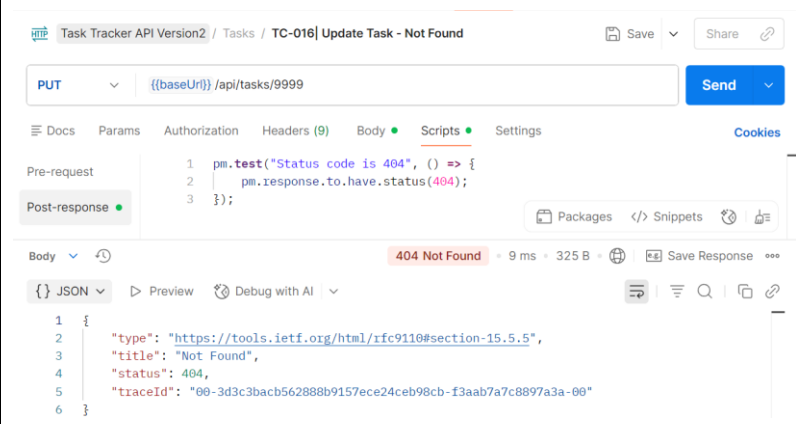
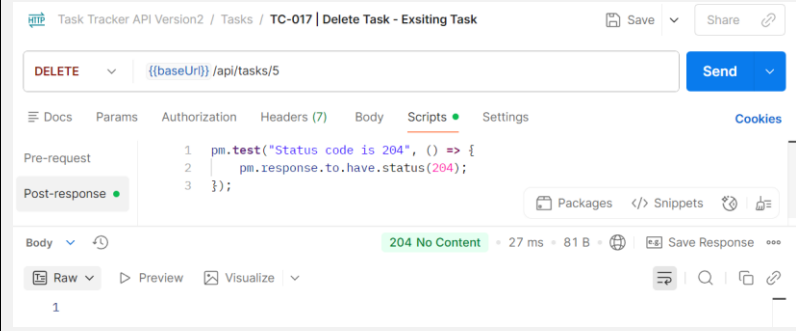
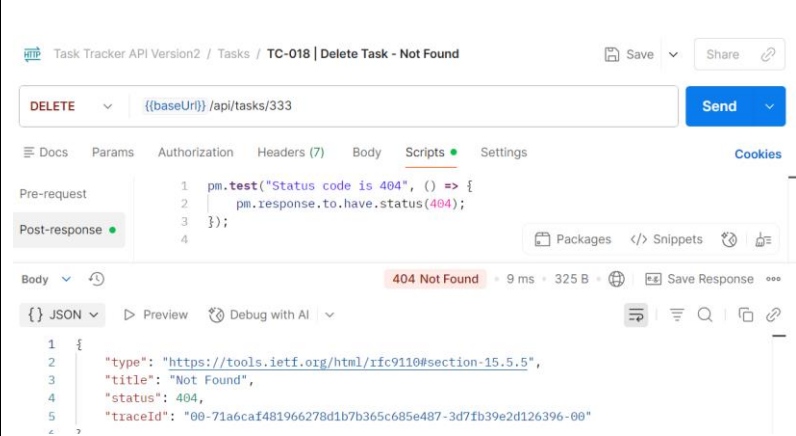
Test ID	Method	Endpoint	Expected Result	Screenshot
TC-007	PUT	/api/users/222	404 Not Found	Task Tracker API Version2 / Users / TC-007 Update User - Not Found PUT {{baseUri}} /api/users/222 Pre-request <pre>1 pm.test("Status code is 404", () => { 2 pm.response.to.have.status(404); 3 });</pre> Post-response Body <pre>1 { 2 "type": "https://tools.ietf.org/html/rfc9110#section-15.5.5", 3 "title": "Not Found", 4 "status": 404, 5 "traceId": "00-7762811e8b76a4959884c382a3ae83b5-7167be05f01f7795-00" 6 }</pre> 404 Not Found · 13 ms · 325 B
TC-008	DELETE	/api/users/8	204 No Content	Task Tracker API Version2 / Users / TC-008 Delete User - Existing User DELETE {{baseUri}} /api/users/8 Pre-request <pre>1 pm.test("Status code is 204", () => { 2 pm.response.to.have.status(204); 3 });</pre> Post-response Body <pre>1</pre> 204 No Content · 93 ms · 81 B
TC-009	DELETE	/api/users/555	404 Not Found	Task Tracker API Version2 / Users / TC-009 Update User - Not Found DELETE {{baseUri}} /api/users/555 Pre-request <pre>1 pm.test("Status code is 404", () => { 2 pm.response.to.have.status(404); 3 });</pre> Post-response Body <pre>1 { 2 "type": "https://tools.ietf.org/html/rfc9110#section-15.5.5", 3 "title": "Not Found", 4 "status": 404, 5 "traceId": "00-8b4e5ff531089b8645c3d53bf72cc09a-fdb4a4abf97a699e-00" 6 }</pre> 404 Not Found · 8 ms · 325 B

6. Test Cases — Tasks Endpoint

The table below lists all test cases executed against the `/api/tasks` endpoint, covering successful requests as well as expected error scenarios (invalid ID, invalid userId).

Test ID	Method	Endpoint	Expected Result	Screenshot
TC-010	GET	/api/tasks	200 OK	 Task Tracker API Version2 / Tasks / TC-010 Get All Tasks GET {{baseUri}}/api/tasks Pre-request <pre>1 pm.test("Status code is 200", () => { 2 pm.response.to.have.status(200); 3 });</pre> Post-response Body <pre>1 [2 { 3 "id": 2, 4 "title": "Updated Task", 5 "isCompleted": false, 6 "userId": 1 7 }, 8 { 9 "id": 4, 10 "title": "Learn CRUD with EF Core", 11 "isCompleted": false, 12 "userId": 1 13 } 14]</pre> 200 OK - 50 ms - 360 B
TC-011	GET	/api/tasks/2	200 OK	 Task Tracker API Version2 / Tasks / TC-011 Get Task By Id - Existing Task GET {{baseUri}}/api/tasks/2 Pre-request <pre>1 pm.test("Status code is 200", () => { 2 pm.response.to.have.status(200); 3 });</pre> Post-response <pre>4 pm.test("Response has task id", () => { 5 pm.expect(pm.response.json()).to.have.pr 6 });</pre> Body <pre>1 { 2 "id": 2, 3 "title": "Updated Task", 4 "isCompleted": false, 5 "userId": 1 6 }</pre> 200 OK - 22 ms - 210 B
TC-012	GET	/api/tasks/99	404 Not Found	 Task Tracker API Version2 / Tasks / TC-012 Get Task By Id - Not Found GET {{baseUri}}/api/tasks/99 Pre-request <pre>1 pm.test("Status code is 404", () => { 2 pm.response.to.have.status(404); 3 });</pre> Post-response Body <pre>1 { 2 "type": "https://tools.ietf.org/html/rfc9110#section-15.5.5", 3 "title": "Not Found", 4 "status": 404, 5 "traceId": "00-71a8ff9456376c861b990422dfdd180-d5de51914e6622a6-00" 6 }</pre> 404 Not Found - 9 ms - 325 B

Test ID	Method	Endpoint	Expected Result	Screenshot
TC-013	POST	/api/tasks	201 Created	Task Tracker API Version2 / Tasks / TC-013 Create Task - Valid Data POST {{baseUrl}}/api/tasks Pre-request <pre>1 pm.test("Status code is 201", () => { 2 pm.response.to.have.status(201); 3 });</pre> Post-response <pre>5 pm.test("Created task has id", () => { 6 pm.expect(pm.response.json()).to.have.property("id"); 7 });</pre> Body <pre>1 { 2 "id": 1005, 3 "title": "Learn CRUD with EF Core", 4 "isCompleted": false, 5 "userId": 1 6 }</pre> 201 Created • 144 ms • 277 B
TC-014	POST	/api/tasks (Invalid UserId)	400 Bad Request	Task Tracker API Version2 / Tasks / TC-014 Create Task - Invalid Data POST {{baseUrl}}/api/tasks Pre-request <pre>1 pm.test("Status code is 400", () => { 2 pm.response.to.have.status(400); 3 });</pre> Post-response <pre>1 { 2 "message": "User does not exist" 3 }</pre> 400 Bad Request • 30 ms • 190 B
TC-015	PUT	/api/tasks/2	204 No Content	Task Tracker API Version2 / Tasks / TC-015 Update Task - Existing Task PUT {{baseUrl}}/api/tasks/2 Pre-request <pre>1 pm.test("Status code is 204", () => { 2 pm.response.to.have.status(204); 3 });</pre> Post-response <pre>1</pre> 204 No Content • 40 ms • 81 B

Test ID	Method	Endpoint	Expected Result	Screenshot
TC-016	PUT	/api/tasks/9999	404 Not Found	
TC-017	DELETE	/api/tasks/5	204 No Content	
TC-018	DELETE	/api/tasks/9999	404 Not Found	

7. Collection Run (Postman Runner)

In addition to executing requests individually, the entire Postman Collection — including both the Users folder and the Tasks folder — was executed together using the Postman Collection Runner. This allows

all requests to run sequentially in one pass and provides a consolidated summary of passed and failed requests/assertions.

The Collection Runner is useful for regression testing: instead of re-running each request manually after a change to the API, the whole collection (or a selected folder) can be re-executed with a single click, and Postman reports the result of every request along with any attached test assertions.

- ☐ Run scope: Entire collection (Users folder + Tasks folder).
- ☐ Iterations: 1 (single pass through all requests).
- ☐ Result: Pass/fail status and response time for each request, summarized at the end of the run.

A screenshot of the Collection Runner summary (showing the results for all requests in both folders) is included below.

The screenshot displays the Postman Collection Runner interface. On the left, a list of 18 test cases is shown, each with a checkbox, a folder icon, a status icon, and a description. All checkboxes are checked. The test cases are numbered 1 through 18. The right panel shows the 'Functional' tab selected, with options to 'Run manually', 'Schedule runs', or 'Automate runs via CLI'. Below this, the 'Run configuration' section includes fields for 'Iterations' (set to 1) and 'Delay' (set to 0 ms). There is also a 'Test data file' section with a 'Select File' button and an 'Advanced settings' link. At the bottom right, there is a red button labeled 'Run Task Tracker API Version2'.

Run Sequence	Deselect All	Select All	Reset
1	☒	☐	☐
2	☒	☐	☐
3	☒	☐	☐
4	☒	☐	☐
5	☒	☐	☐
6	☒	☐	☐
7	☒	☐	☐
8	☒	☐	☐
9	☒	☐	☐
10	☒	☐	☐
11	☒	☐	☐
12	☒	☐	☐
13	☒	☐	☐
14	☒	☐	☐
15	☒	☐	☐
16	☒	☐	☐
17	☒	☐	☐
18	☒	☐	☐

Functional Performance

Choose how to run your collection

☒ Run manually
Run this collection in the Collection Runner.

☐ Schedule runs
Periodically run collection at a specified time on the Postman Cloud.

☐ Automate runs via CLI
Configure CLI command to run on your build pipeline.

Run configuration

Iterations ⓘ
1

Delay ⓘ
0 ms

Test data file ⓘ
Only JSON and CSV files are accepted.
Select File

> Advanced settings

Run Task Tracker API Version2

Task Tracker API Version2 - Run results
Run Again
+ New Run
Automate Run
Share

Ran today at 02:50:41 PM · [View all runs](#)

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	Local Development	1	2s 675ms	21	0	7 ms

RUN SUMMARY
[View Results](#)

1

- GET TC-001 | Get All Users 2 | 0
- GET TC-002 | GET User By Id - Existing User 0 | 0
- GET TC-003 | GET User By Id - Not Found 1 | 0
- POST TC-004 | Create User - Valid Data 2 | 0
- POST TC-005 | Create User - Invalid Data 1 | 0
- PUT TC-006 | Update User - Existing User 1 | 0
- PUT TC-007 | Update User - Not Found 1 | 0
- DELETE TC-008 | Delete User - Existing User 1 | 0
- DELETE TC-009 | Update User - Not Found 1 | 0
- GET TC-010 | Get All Tasks 1 | 0
- GET TC-011 | Get Task By Id - Existing Task 2 | 0
- GET TC-012 | Get Task By Id - Not Found 1 | 0
- POST TC-013 | Create Task - Valid Data 2 | 0
- POST TC-014 | Create Task - Invalid Data 1 | 0
- PUT TC-015 | Update Task - Existing Task 1 | 0
- PUT TC-016 | Update Task - Not Found 1 | 0
- DELETE TC-017 | Delete Task - Existing Task 1 | 0
- DELETE TC-018 | Delete Task - Not Found 1 | 0

8. Test Summary

A total of 18 test cases were executed across the Users and Tasks endpoints, covering all four core HTTP methods (GET, POST, PUT, DELETE) as well as both success and failure scenarios.

Total Test Cases	18
Users Endpoint Tests	9
Tasks Endpoint Tests	9
Success Scenarios	10
Error Scenarios (4	8

8. Conclusion

All planned test cases were executed successfully using Postman, with results matching the expected HTTP status codes defined for each scenario. The API correctly handles both valid requests — returning

appropriate success codes (200, 201, 204) — and invalid or non-existent resource requests — returning the correct error codes (400, 404).

This confirms that the Users and Tasks endpoints of the API meet the expected functional requirements for basic CRUD operations and input validation. Screenshot evidence for each test case is provided in the tables above and should be inserted at the indicated placeholders.